VendorShield MANUAL (A-Z)

TPRM + GRC mini-platform: build log, results, interview prep. SAMPLE demo data. Local only.

#1. Objective

• Goal: prove hands-on skill for a Cyber Security and Technology Analyst role via a local-only Third-Party Risk Management (TPRM) + GRC mini-platform called VendorShield.

• What it proves: vendor due diligence, scored assessments, document review, risk register, GRC workflow, IAM, SOC basics, business continuity, ITGC, reporting, and self-VAPT.

• Rules kept: local dev only (PostgreSQL + SQLite tests, no Docker/cloud), simple readable code with WHY comments, nothing faked (all demo rows labelled SAMPLE), built in 11 phases.

#2. Tech stack

• Backend: Python, FastAPI, SQLAlchemy, Alembic, PostgreSQL (dev/demo) + SQLite memory (pytest).

• Frontend: React (Vite 5), Tailwind CSS, Recharts. Auth: JWT access (15 min) + refresh (7 days, jti rotation), bcrypt, TOTP MFA via pyotp.

• Analysis: pandas + Matplotlib. Optional AI: Gemini stub OFF by default (draft + reviewer-must-confirm + hallucination/privacy note).

• Quality: 61 pytest tests, Bandit, pip-audit, npm audit, Postman collection.

#3. What was built (Phases 1-11)

• P1 structure/schema/seed: 16 tables, Alembic, 12 SAMPLE vendors, 25 questions, 4 users (Password123!), docs/risks/SOC samples.

• P2 auth: login/refresh/MFA setup-verify, require_roles RBAC, SoD approvals (2nd Admin), append-only audit, 5-fails/5min rate limit, strict CORS + security headers.

• P3 vendors + assessments: CRUD, lifecycle allowlist (Reviewer to Assessment/Review, Admin approves), tier freq 6/12/24, delete via 2nd Admin; questionnaire with evidence rule, weighted score + levels, remediation rules.

• P4 documents: secure upload (allowlist, magic-header, 10MB, random names, non-public dir), Valid/Expiring/Expired flags, review, AI stub disabled.

• P5 risks: LxI 1-5, residual<=inherent with controls proof, 5x5 heatmap, overdue flags, remediations, exception approvals.

• P6 governance: policies + versions, ISO/NIST coverage %, BCP gaps (tier1/High + missing/untested/no-exit), ITGC Pass-needs-evidence + config-change log.

• P7 SOC: events auto-logged (failed_login, login_success, suspicious_upload, privilege_change), 3 rules (brute-force/new-IP/probe) with dedupe, triage, stepwise incidents + timeline, SAMPLE-only simulator.

• P8 frontend: 13 role-filtered pages, JWT wrapper, vite proxy, Recharts dashboard.

• P9 reporting: dashboard JSON, vendor + executive PDFs (BytesIO), pandas charts + insights.md.

• P10 AppSec: Bandit clean, PyJWT 2.15.1 fix, esbuild fix, braces accepted dev-only, 6 extra attack tests, VAPT_REPORT + SECURITY.

• P11 docs: BRD, user stories, methodology, architecture, presentation outline, interview guide, README.

#4. Scoring methodology

• Assessment: Yes=1, Partial=0.5, No=0, N/A excluded. Score = sum(w*v)/sum(w answered)*100. Levels: >=85 Low, 60-84 Medium, 40-59 High, <40 Critical.

• Example: weights 3/1/2 with Yes/No/Partial = (3+0+1)/6 = 66.67 Medium.

• Risk: inherent = likelihood x impact (1-25). Low 1-4, Medium 5-9, High 10-15, Critical 16-25. Residual starts = inherent; lowering needs controls text.

• Tier cadence: T1=6mo, T2=12mo, T3=24mo. BCP gap = critical vendor with no plan / no exit / never tested / test older than 1 year.

#5. API list (all RBAC-enforced)

• Auth: POST /api/v1/auth/login | refresh | mfa/setup | mfa/verify.

• Users: GET/POST /users, GET /users/{id}, POST role-request | revoke | confirm. Approvals: GET /approvals, POST /approvals/{id}/decide. Audit: GET /audit-logs (no edit/delete).

• Vendors: GET/POST /vendors, GET/PATCH /vendors/{id}, POST transition, POST delete-request.

• Assessments: GET /questions, POST /vendors/{id}/assessments, GET /assessments/{id}, PUT answers, POST submit.

• Documents: POST upload, GET list(?status=), GET/{id}, PATCH review, POST ai-summary/ai-confirm.

• Risks: GET/POST /risks, GET/PATCH /risks/{id}, GET heatmap, remediations CRUD, POST exceptions.

• Governance: policies CRUD + versions, frameworks controls/map/coverage, continuity + gaps report, itgc + config-changes.

• SOC: security-events, detection/run, alerts + triage, incidents + transition + notes.

• Reports: GET dashboard, GET vendor/{id}.pdf, GET executive.pdf.

#6. How to run and test

• Backend: cd backend; copy .env.example .env; pip install -r requirements.txt; create DB vendorshield (Postgres 18, user postgres); python -m app.db.seed; python -m pytest tests/ -q (61 passed); uvicorn app.main:app --reload (:8000).

• Useful: python -m scripts.check_expiry; python -m scripts.simulate_attacks (SAMPLE only); python -m scripts.analysis (writes backend/reports/*.png + insights.md).

• Frontend: cd frontend; npm install --legacy-peer-deps; npm run dev (:5173); npm run build (must succeed).

• Demo logins (SAMPLE): admin / reviewer / auditor / vendor1 @example.com with Password123!. Admin TOTP: mfa/setup then verify.

#7. Outcomes and results

• 61 pytest tests green (auth 7, RBAC/attacks 8, vendors 7, assessments 5, docs 5, risks 6, governance 4, monitoring 5, reports 4, security-extra 6, phase1 4).

• Bandit: No issues (2277 LOC). Raw-SQL grep: no hits (ORM only). PyJWT fixed to 2.15.1. npm: esbuild fixed, braces accepted dev-only.

• Seed verified on Postgres: 12 vendors, 4 users, 25 questions; expiry script finds 1 EXPIRING + 1 EXPIRED SAMPLE; simulator creates 3 alerts; analysis writes 3 charts + insights.

• Deliverables: React UI (13 pages), dashboard + 2 PDFs, Postman collection (40+ requests), 7 docs + SECURITY.md.

#8. Interview questions and short answers

• Q: PostgreSQL vs SQLite? A: Postgres for demo realism; SQLite memory for speed. Avoid Postgres-only column types.

• Q: Tier drives review? A: tier_frequency() in services; routes stay thin so API/tests/docs share one function.

• Q: Audit append-only? A: no PUT/DELETE routes; fields actor/action/entity/IP/old-new JSON.

• Q: Access vs refresh? A: 15-min access limits theft; 7-day refresh with jti rotation; type claim stops confusion.

• Q: Same Admin self-approve? A: 403 SoD; second Admin must decide.

• Q: 401 vs 403 vs 429? A: 401 bad identity/token, 403 valid identity but forbidden role/scope, 429 too many attempts.

• Q: Vendor answers but not submits? A: SoD - vendor provides evidence, reviewer rates.

• Q: 66.67 Medium? A: (3*1+1*0+2*0.5)/6; N/A excluded from denominator.

• Q: Block hard delete with history? A: keeps audit trail; offboard instead.

• Q: Renamed exe to pdf? A: extension + MIME + magic-header checks block it; suspicious_upload event logged.

• Q: Random filenames? A: stops guessing/traversal; stored outside public path.

• Q: AI must stay unconfirmed? A: hallucination + data-privacy risks; human confirms.

• Q: Residual<=inherent? A: reduction needs controls proof, else 400.

• Q: Heatmap cell L4xI5? A: row impact 5, col likelihood 4; 20 = Critical.

• Q: Exception approve flips to accepted not closed? A: accepted = conscious expiry-bound risk, still visible.

• Q: Coverage %? A: mapped controls/total; unmapped listed as gaps.

• Q: Reviewer cannot Approve policy? A: only Admin approves (governance).

• Q: 400-day test is a gap? A: yes, stale test means unproven recovery even if plan exists.

• Q: created_at needed? A: yes for 5-in-5-min windows; added to SOC tables.

• Q: Dedupe key? A: rule+detail while status New stops storms.

• Q: Stepwise incidents? A: stops skipping containment; Detected-Triaged-Contained-Eradicated-Recovered-LessonsLearned.

• Q: Only hide UI by role? A: never; backend re-checks every endpoint (defence in depth).

• Q: JWT in localStorage risk? A: XSS theft; prod should use httpOnly cookies.

• Q: Vite proxy? A: avoids CORS in dev; backend stays strict to :5173.

• Q: Completion %? A: vendors with submitted assessment / total vendors.

• Q: BytesIO PDFs? A: no temp files, no cleanup/leak risk.

• Q: Trend x-axis id order? A: assessments lack created_at; honestly noted, fix = add timestamp.

• Q: Why bump PyJWT? A: known CVEs fixed in 2.15.1; exp/type/jti stop forgery/replay/confusion.

• Q: Braces accepted? A: fix needs tailwind 4 breaking change; local dev-only risk, documented.

• Q: Traversal 201 safe? A: stored name is random hex, original path never used.

#9. Honest limitations and next steps

• Local demo only: no TLS/WAF/prod hardening. JWT in localStorage (use httpOnly cookies in prod).

• perm_denied auto-logging is manual/simulation-fed; global 403 middleware deferred.

• Assessments lack created_at (trend = id order). Seed SAMPLE password documented demo-only.

• Next: httpOnly cookies, assessment timestamps, 403 middleware, tailwind 4 upgrade, Postgres-backed refresh store, TLS + WAF in staging.

Appendix: module status

Module	Status
Vendors/lifecycle	done, 7 tests
Assessments/scoring	done, 5 tests
Documents/AI stub	done, 5 tests
Risks/heatmap	done, 6 tests
Policies/frameworks/BCP/ITGC	done, 4 tests
SOC/incidents	done, 5 tests
Frontend (13 pages)	done, build green
Dashboard/PDFs/analysis	done, 4 tests
VAPT/SECURITY	done, 6 extra tests
Docs (7 + README)	done